

The background is a solid blue color with a complex, abstract pattern of white and light blue lines and dots. These lines resemble circuit traces or neural network connections, with some lines ending in small circles. The pattern is more dense on the left side and becomes more sparse towards the right.

O QUE SÃO E COMO FUNCIONAM AS

Redes Neurais Convolucionais (CNNs)

Prof. Dr. Luis Alvaro de Lima Silva
Mestranda Gabrieli S. J. Müller

Pré Requisitos

Familiaridade com os fundamentos de Redes Neurais

Familiaridade com a ideia de Backpropagation

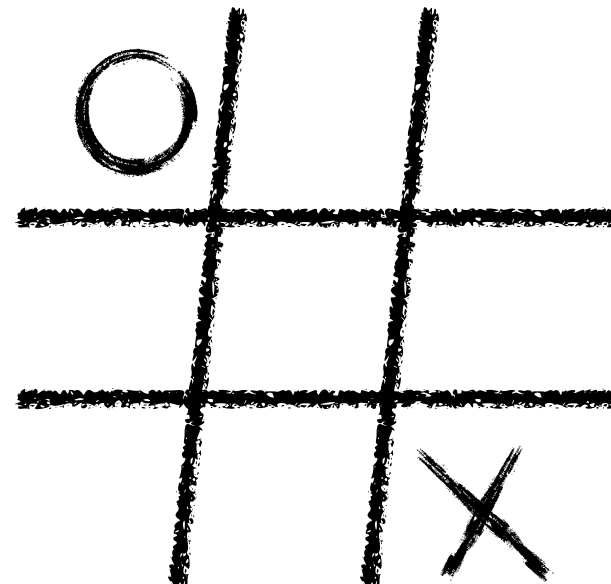
Função de Ativação (ReLU)

Situação inicial

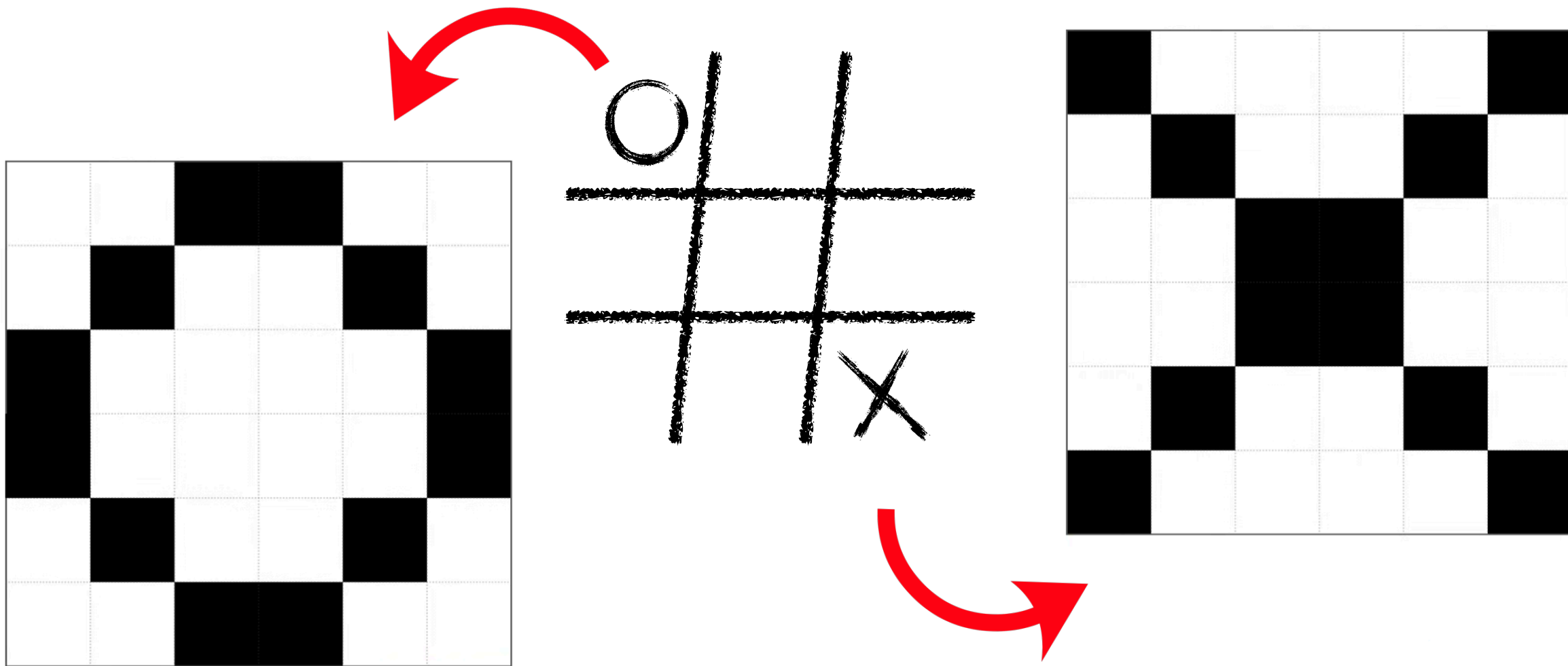
Vamos imaginar que um **computador** está jogando o jogo-da-velha contra um **usuário humano**.

Cada vez que esse **usuário humano** inicia a partida ele escolhe aleatoriamente entre “O” ou “X”.

Em seu turno, o **computador** deve reconhecer qual foi o símbolo usado pelo **usuário** para só então poder jogar.



Cada imagem é uma matriz de pixels



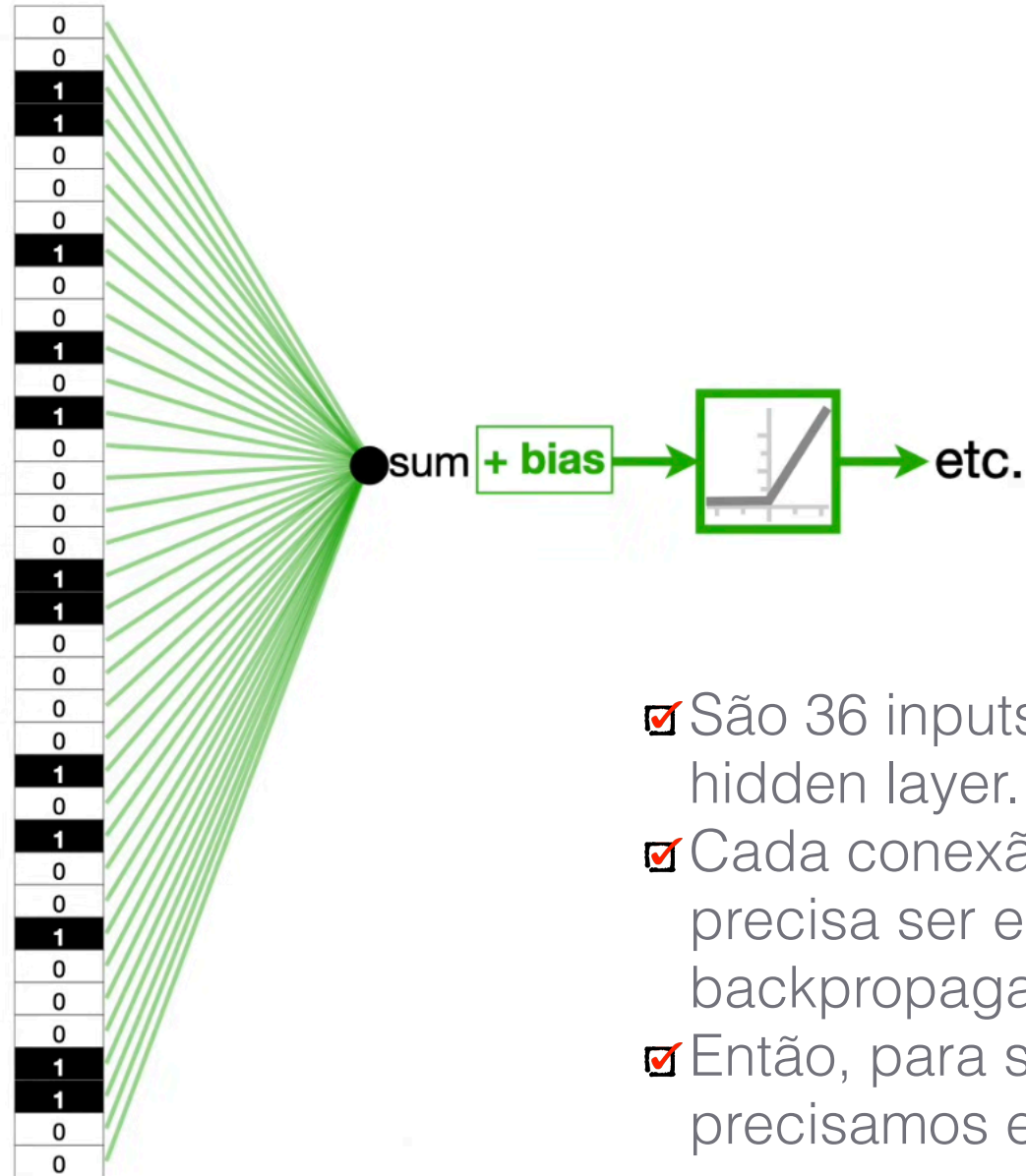
...E, nesse caso, cada pixel pode ser representado por 0 ou 1

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Obtivemos uma matriz 6x6.
Poderíamos tornar cada pixel num *input* e rodar uma MLP, correto?

Ficaria assim:

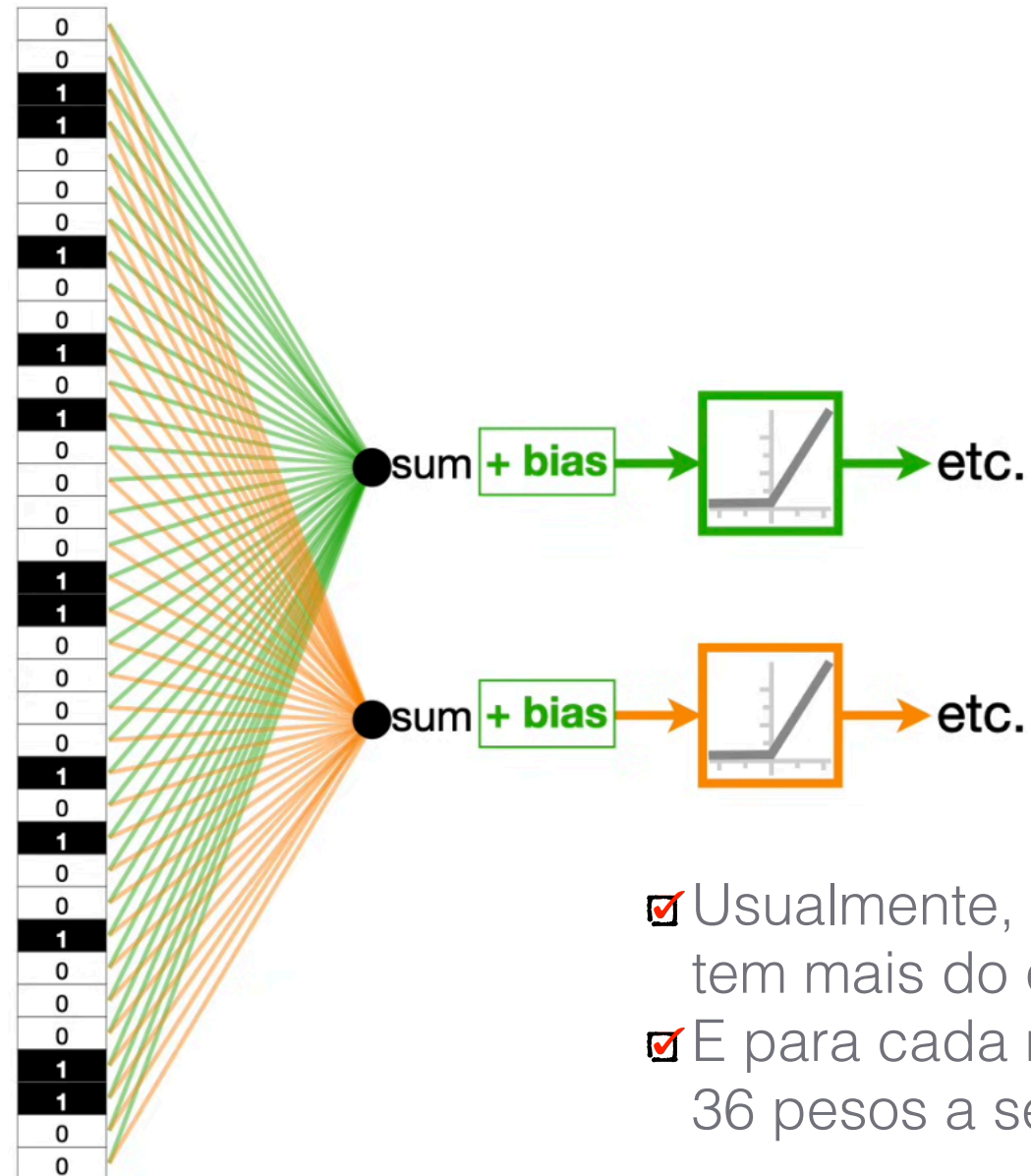
0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0



- ☑ São 36 inputs conectados a uma hidden layer.
- ☑ Cada conexão tem um peso, que precisa ser estimado com backpropagation
- ☑ Então, para se conectar a esse nó, precisamos estimar 36 pesos

No entanto...

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0



- ✓ Usualmente, a primeira hidden layer tem mais do que apenas um node
- ✓ E para cada nó adicional são mais 36 pesos a serem estimados

O problema de utilizar MLP para imagens

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

A imagem original é uma matriz bem pequena, de 6x6. E em preto e branco.

Agora imagine utilizar MLP em imagens com dimensões na casa dos megapixels...

1º problema de utilizar MLP para imagens: Explosão de Parâmetros

Em uma rede totalmente conectada (MLP), uma imagem de apenas 1000×1000 pixels (com 3 canais RGB) resulta em uma camada de entrada de 3.000.000 de neurônios.

Conectar essa entrada a uma única camada oculta de 1000 neurônios exigiria:

$$3.000.000 \times 1.000 = 3.000.000.000 \text{ pesos (3 bilhões de parâmetros)}$$

Portanto esse não é um meio muito escalável.

2º problema de utilizar MLP para imagens: Variância por Translação

Não é evidente que uma MLP vá performar bem se o desenho estiver posicionado em outro lugar da imagem, mesmo que só um pixel ao lado da posição de treino.

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0



0	0	0	1	1	0
0	0	1	0	0	1
0	1	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	1
0	0	0	1	1	0

3º problema de utilizar MLP para imagens: Correlação Espacial

Mesmo imagens mais “complicadas” tendem a ter correlação entre os pixels vizinhos.

Nesta imagem, podemos afirmar que pixels da cor marrom tendem a estarem próximos de outros pixels da cor marrom, enquanto pixels da cor branca tendem a estarem mais próximos de outros pixels brancos.

É útil que levemos isso em consideração e nos aproveitemos das correlações existentes entre pixels.



Como as Redes Neurais Convolucionais resolvem esses problemas?

- I. Elas reduzem o número de input nodes;
- II. Implementam Invariância por Translação, isto é, consideram que um padrão é o mesmo independentemente do lugar onde se encontram na imagem;
- III. Tomam vantagem do fato de que pixels vizinhos tendem a ser correlacionados.

Como funcionam as CNNs?

Os componentes de uma Rede Neural Convolutacional trabalham juntos para extrair características importantes da imagem, as features (ex: bordas verticais, texturas, padrões, etc.), que resultam em um **feature map**.

Esses componentes são:

- I. Convoluções (Kernels ou Filtros)
- II. Pooling

Vamos entender mais adiante o que são cada deles.

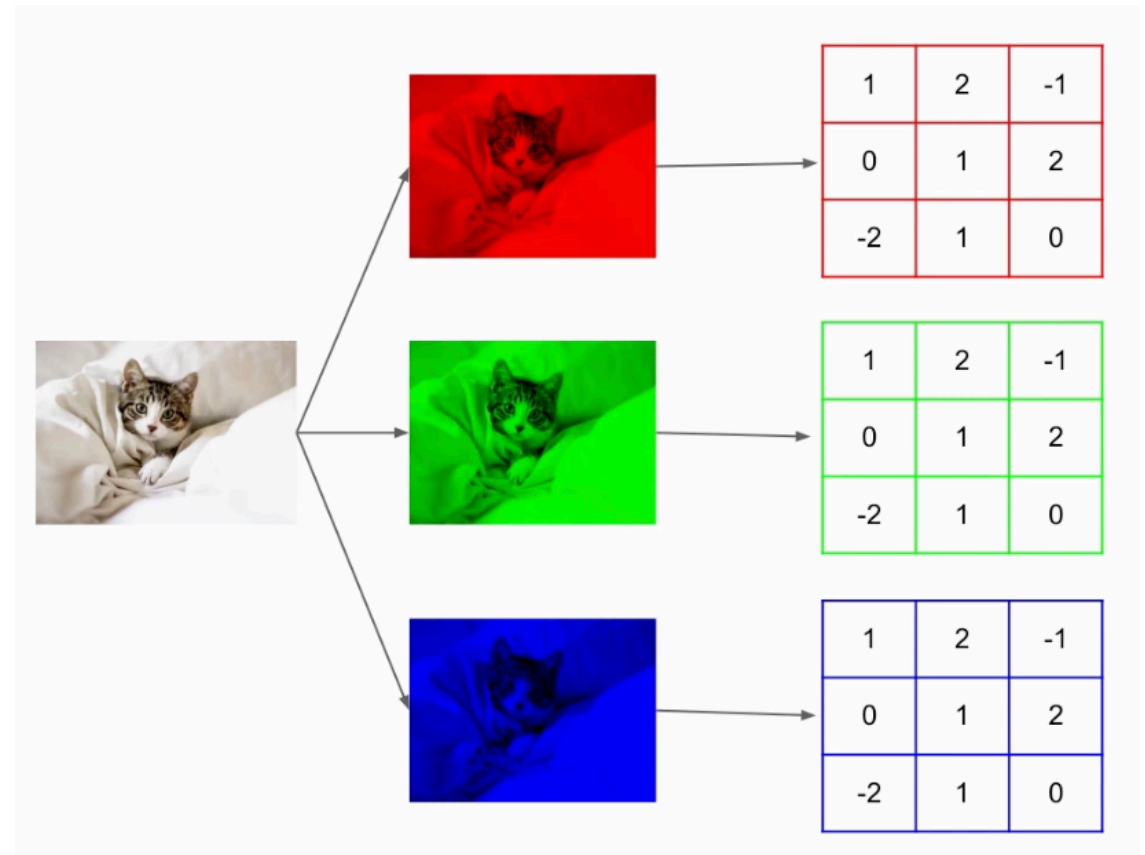
Como funcionam as CNNs?

As entradas serão matrizes bidimensionais (para imagens em preto e branco) ou tridimensionais (para imagens coloridas).

No caso de matrizes tridimensionais, a profundidade será determinada pela quantidade de canais de cores.

Normalmente as imagens utilizam 3 canais: RGB, com valores de 0 a 255.

Por enquanto, vamos trabalhar com exemplos bidimensionais.



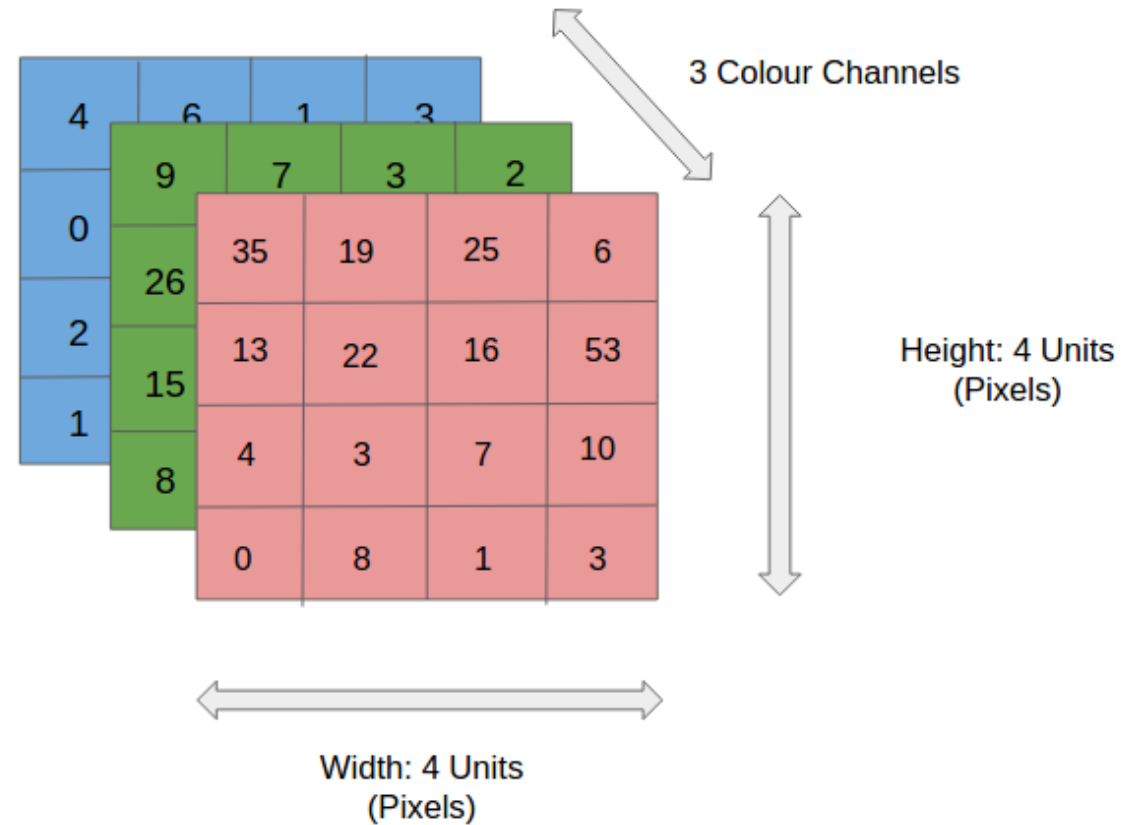
Como funcionam as CNNs?

As entradas serão matrizes bidimensionais (para imagens em preto e branco) ou tridimensionais (para imagens coloridas).

No caso de matrizes tridimensionais, a profundidade será determinada pela quantidade de canais de cores.

Normalmente as imagens utilizam 3 canais: RGB, com valores de 0 a 255.

Por enquanto, vamos trabalhar com exemplos bidimensionais.



Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

I. A primeira coisa que a CNN faz é aplicar um **filtro** (kernel) na matriz de input.

* O programador define o tamanho e quantidade de kernels;

* A intensidade de cada pixel no filtro (i.e. os valores da matriz) é determinada por backpropagation

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

- I. O filtro é sobreposto na matriz de input
- II. E então cada pixel sobreposto é multiplicado

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

$$\begin{aligned} & (0 \times 0) + (0 \times 0) + (1 \times 1) \\ & + (0 \times 0) + (1 \times 1) + (0 \times 0) \\ & + (1 \times 1) + (0 \times 0) + (0 \times 0) \end{aligned}$$

= 3

Essa multiplicação de matrizes resulta no que chamamos de produto escalar.

O nome “convolução” vem da ideia de que, após o cálculo do produto escalar, o Kernel está convoluto com a matriz de input.

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

$$(0 \times 0) + (0 \times 0) + (1 \times 1) \\ + (0 \times 0) + (1 \times 1) + (0 \times 0)$$

Essa multiplicação de matrizes resulta no que chamamos de produto escalar.

O nome “convolução” vem da ideia de que, após o cálculo do produto escalar, o Kernel está convoluto com a matriz de input.

 **convoluto**

Convolutio significa algo que está enrolado, dobrado sobre si mesmo ou envolvido em formato cilíndrico. A palavra é muito utilizada na **botânica** (ex: folhas que se dobram em

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

$$\begin{aligned} & (0 \times 0) + (0 \times 0) + (1 \times 1) \\ & + (0 \times 0) + (1 \times 1) + (0 \times 0) \\ & + (1 \times 1) + (0 \times 0) + (0 \times 0) \end{aligned}$$

Produto + Bias

= 3

+ -2

Feature map

1			

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

$$\begin{aligned} & (0 \times 0) + (1 \times 0) + (1 \times 1) \\ & + (1 \times 0) + (0 \times 1) + (0 \times 0) \\ & + (0 \times 1) + (0 \times 0) + (0 \times 0) \end{aligned}$$

Produto + Bias

= 1

+ -2

Feature map

1	-1		

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

$$\begin{aligned} & (1 \times 0) + (1 \times 0) + (0 \times 1) \\ & + (0 \times 0) + (0 \times 1) + (1 \times 0) \\ & + (0 \times 1) + (0 \times 0) + (0 \times 0) \end{aligned}$$

Produto + Bias

= 0

+ -2

Feature map

1	-1	-2	

Voltando ao símbolo do jogo-da-velha...

Input 6x6

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Kernel 3x3

0	0	1
0	1	0
1	0	0

Final feature map

1	-1	-2	-1
-1	-2	-1	-2
-2	-1	-2	-1
-1	-2	-1	1

Voltando ao símbolo do jogo-da-velha...

Lembrando: o feature map nos ajuda a tirar vantagem de quaisquer correlações que possam haver na imagem.

E, para constar, o tamanho do feature map é dado por:

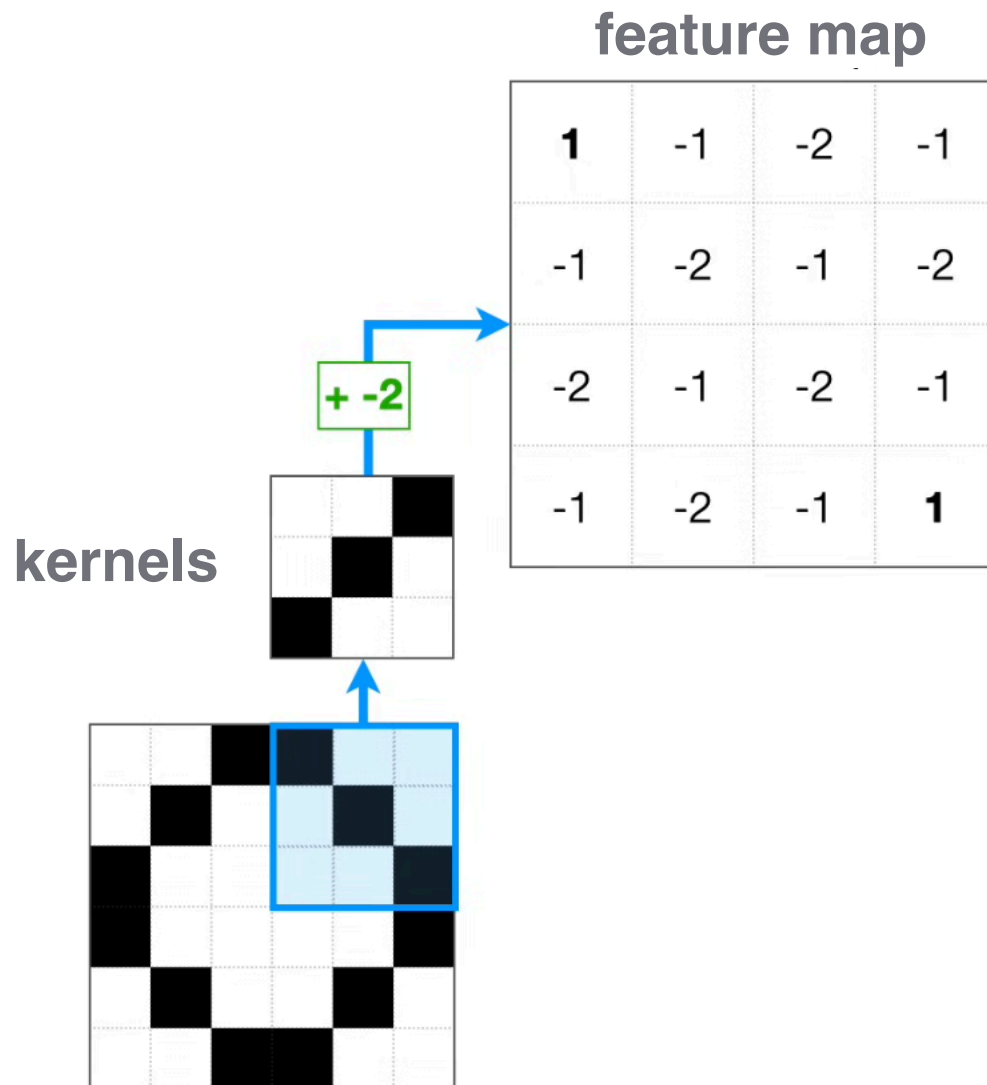
$$O = [(I - F + 2P) / S] + 1$$

- **I**: Dimensão da imagem de entrada (altura/largura)
- **F**: Tamanho do filtro (Kernel size)
- **P**: Quantidade de preenchimento de zeros (Padding) -> vamos ver
- **S**: Passo do filtro (Stride)

Final feature map

1	-1	-2	-1
-1	-2	-1	-2
-2	-1	-2	-1
-1	-2	-1	1

Review



Até aqui executamos uma camada convolucional com as seguintes configurações:

padding: 0

stride: 1

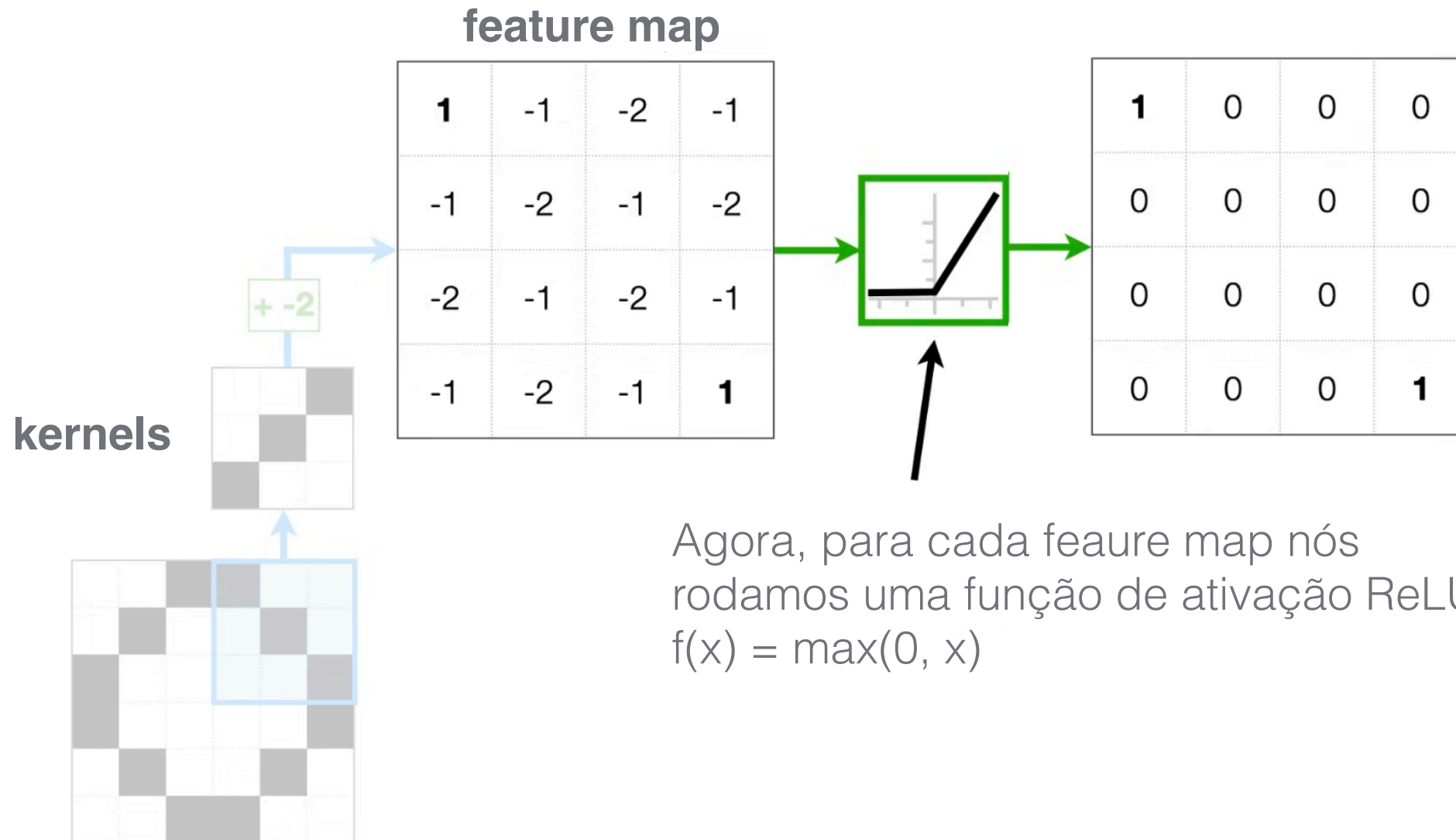
numero de kernels: 1

dimensão dos kernels: 3x3

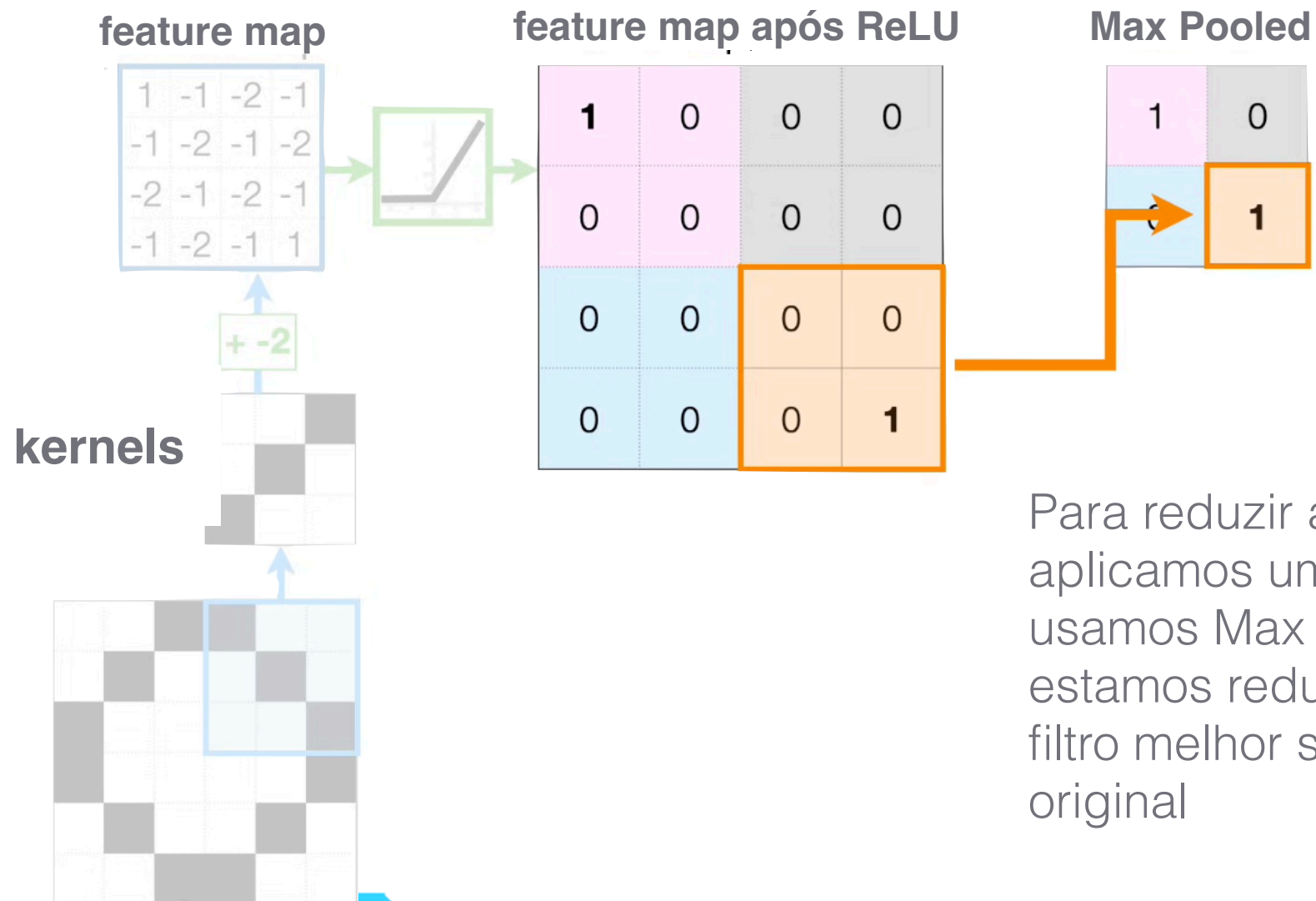
número de canais: 1

Mas na prática nós utilizamos múltiplos kernels e, portanto, no final, temos vários feature maps empilhados.

Camada ReLU



Camada de Pooling



Para reduzir a dimensionalidade, aplicamos um método de pooling (aqui usamos Max Pooling), dessa forma estamos reduzindo as regiões onde o filtro melhor se encaixou com a imagem original

Feature Learning

Esse processo que fizemos até aqui (Convoluções + ReLU + Pooling) pode ser feito diversas vezes, o quanto for necessário para que o modelo aprenda a identificar objetos ou seres na imagem.

Algumas arquiteturas sugerem, por exemplo:

Conv
ReLU
Pooling

Conv
ReLU
Pooling

...

outras:

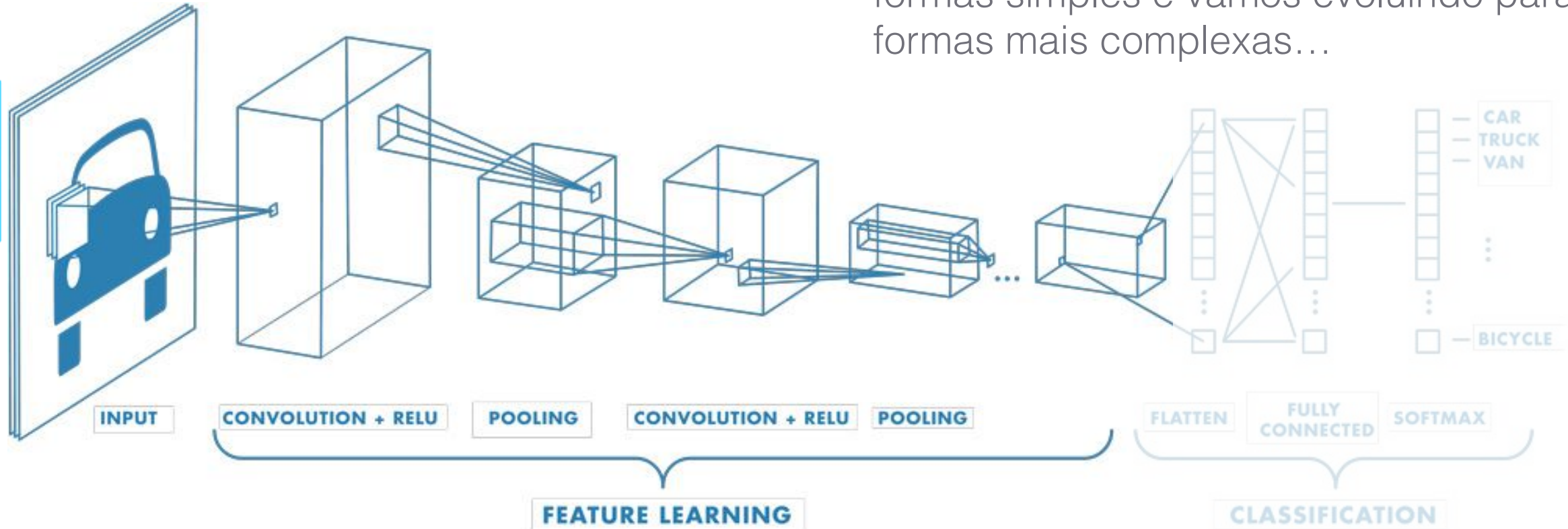
Conv
ReLU

Conv
ReLU
Pooling

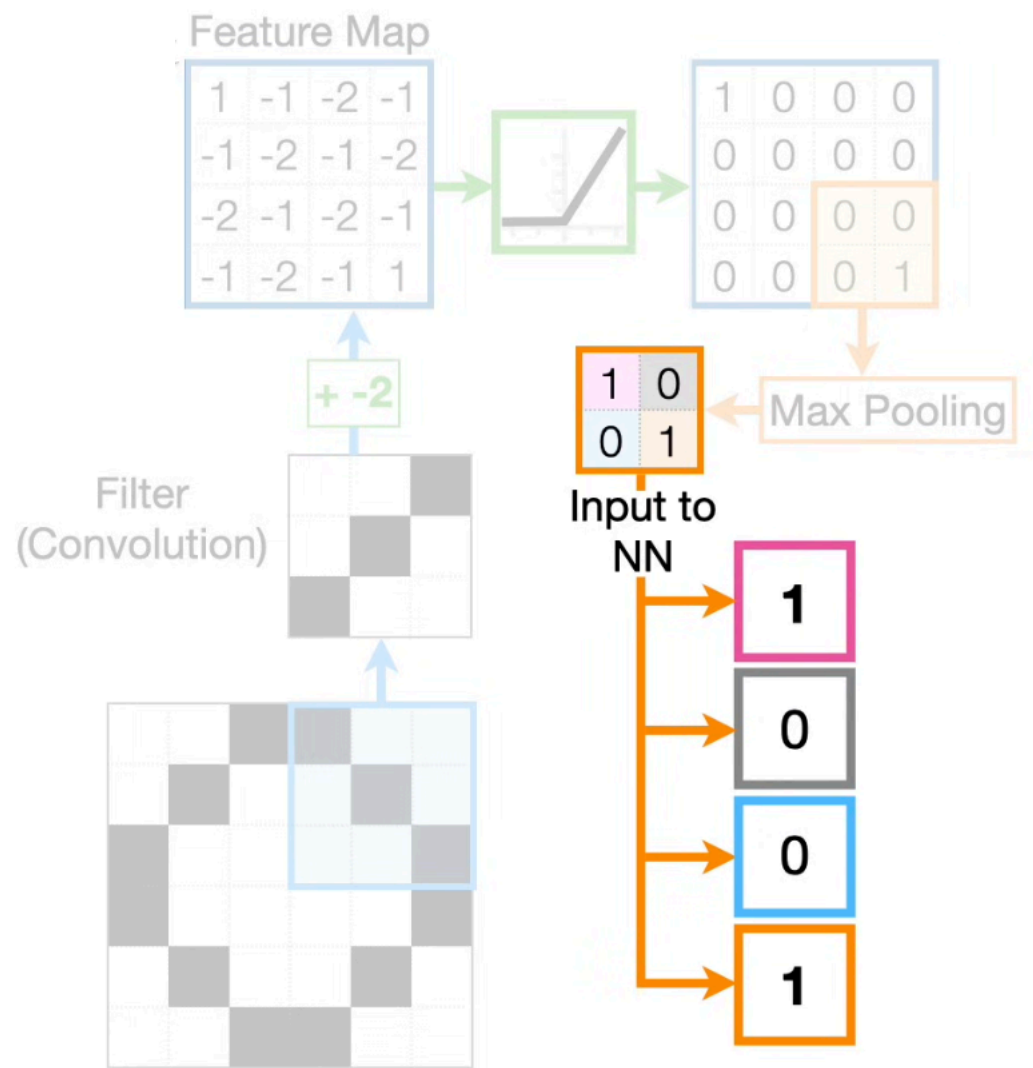
...

Feature Learning

Nas primeiras iterações a rede aprende formas simples e vamos evoluindo para formas mais complexas...

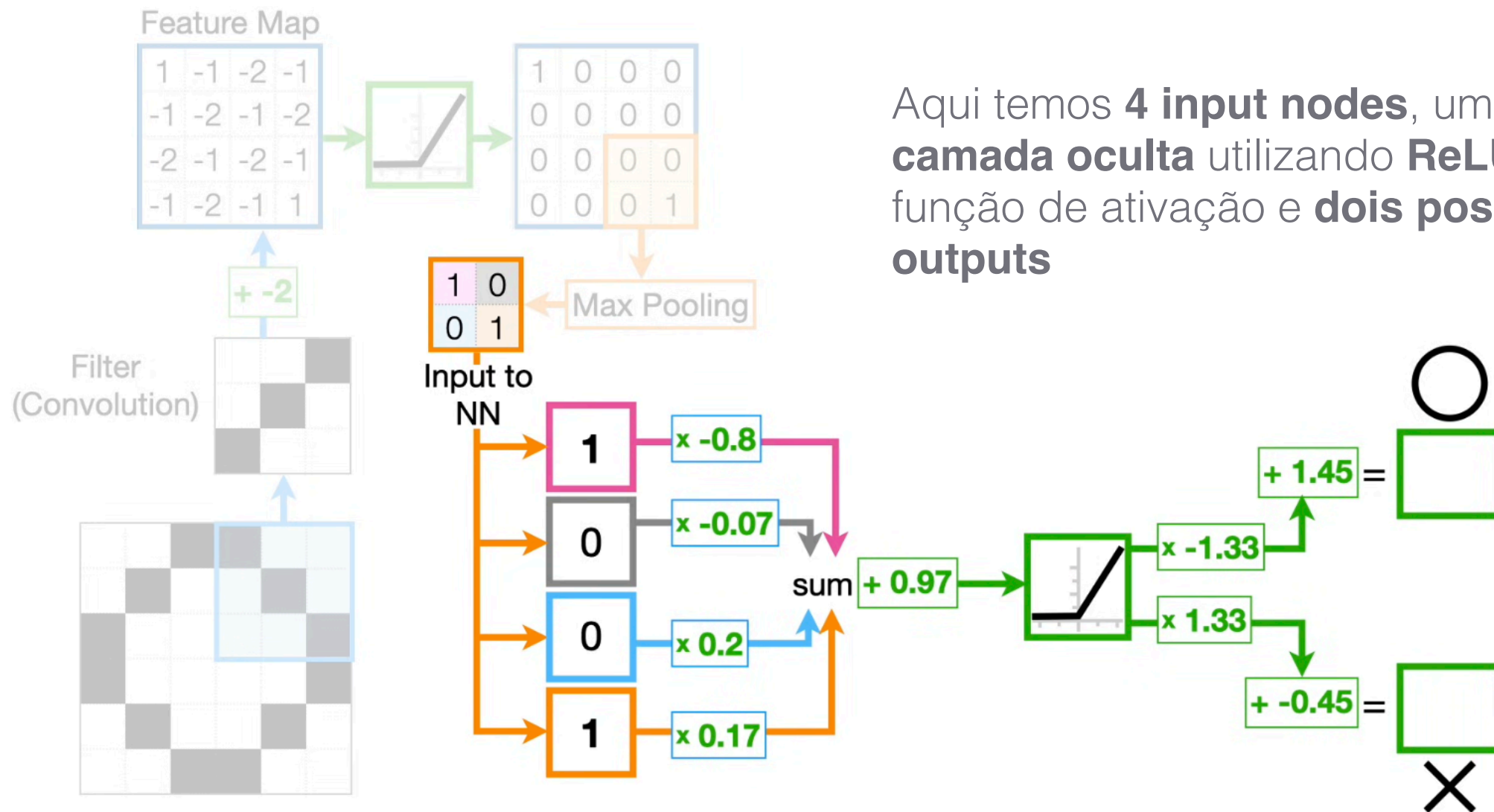


Camadas de Classificação da Rede Neural



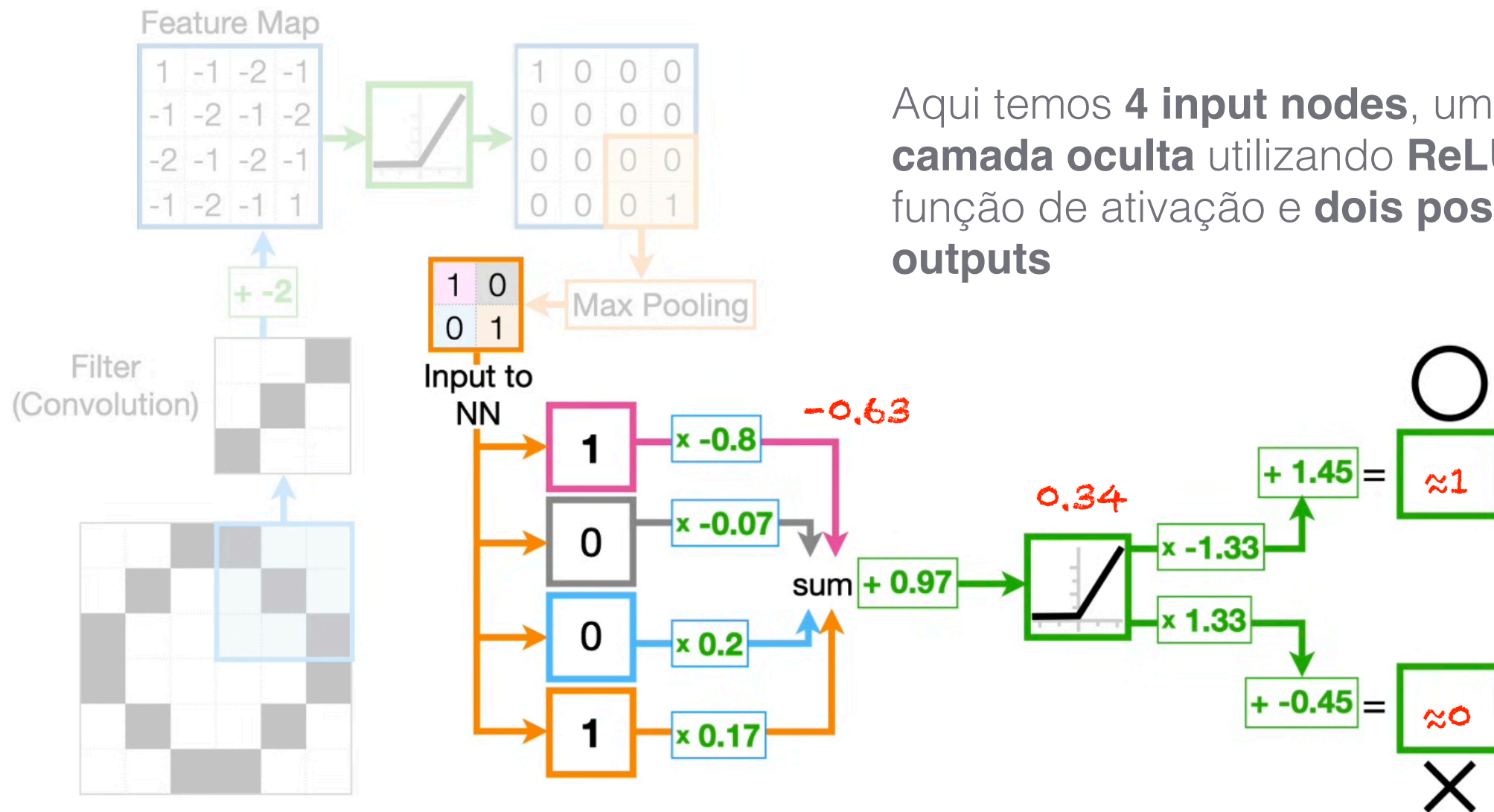
Agora, podemos rodar uma Rede Neural utilizando os resultados do Max Pooling como Input Nodes

Camadas de Classificação da Rede Neural



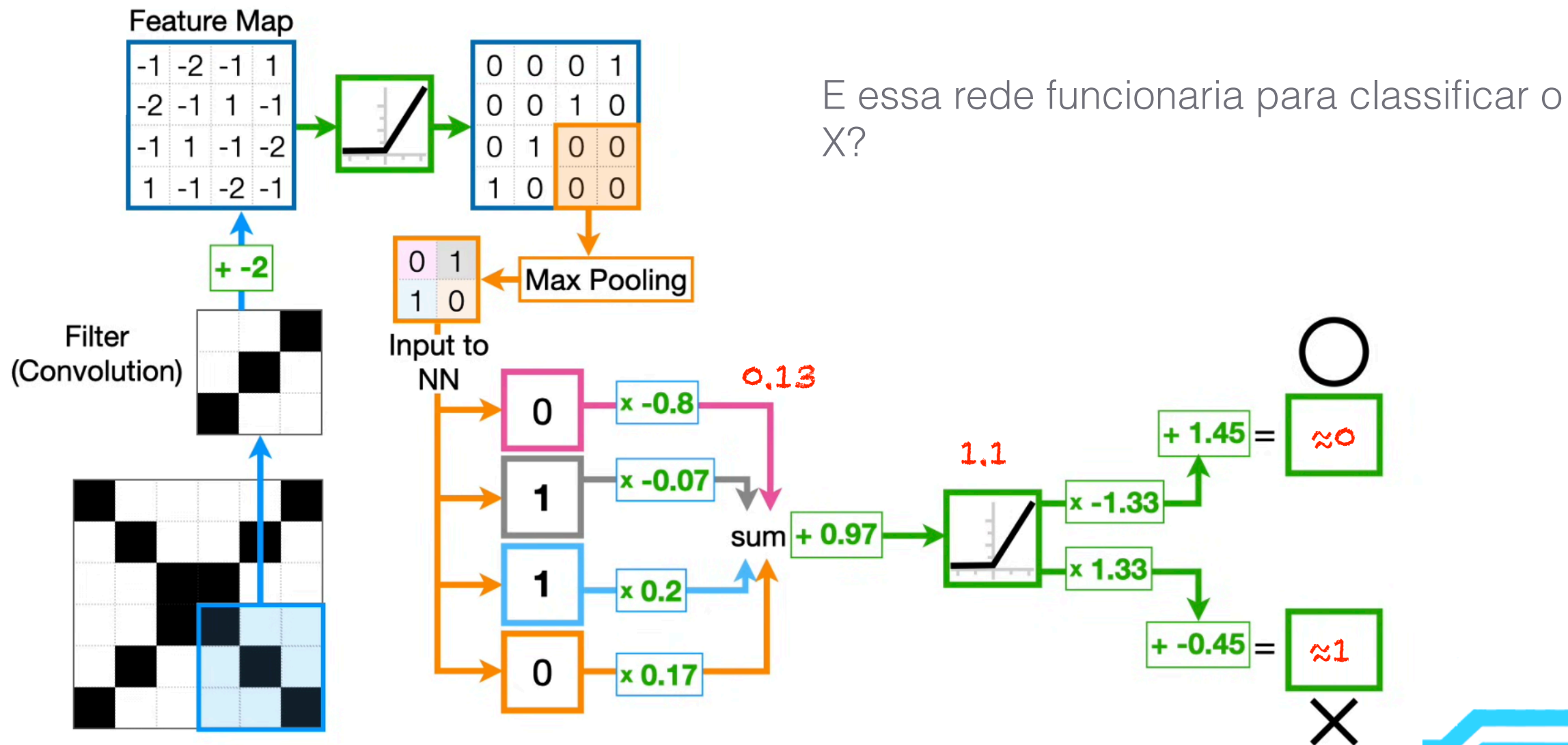
Aqui temos **4 input nodes**, uma única **camada oculta** utilizando **ReLU** como função de ativação e **dois possíveis outputs**

Camadas de Classificação da Rede Neural

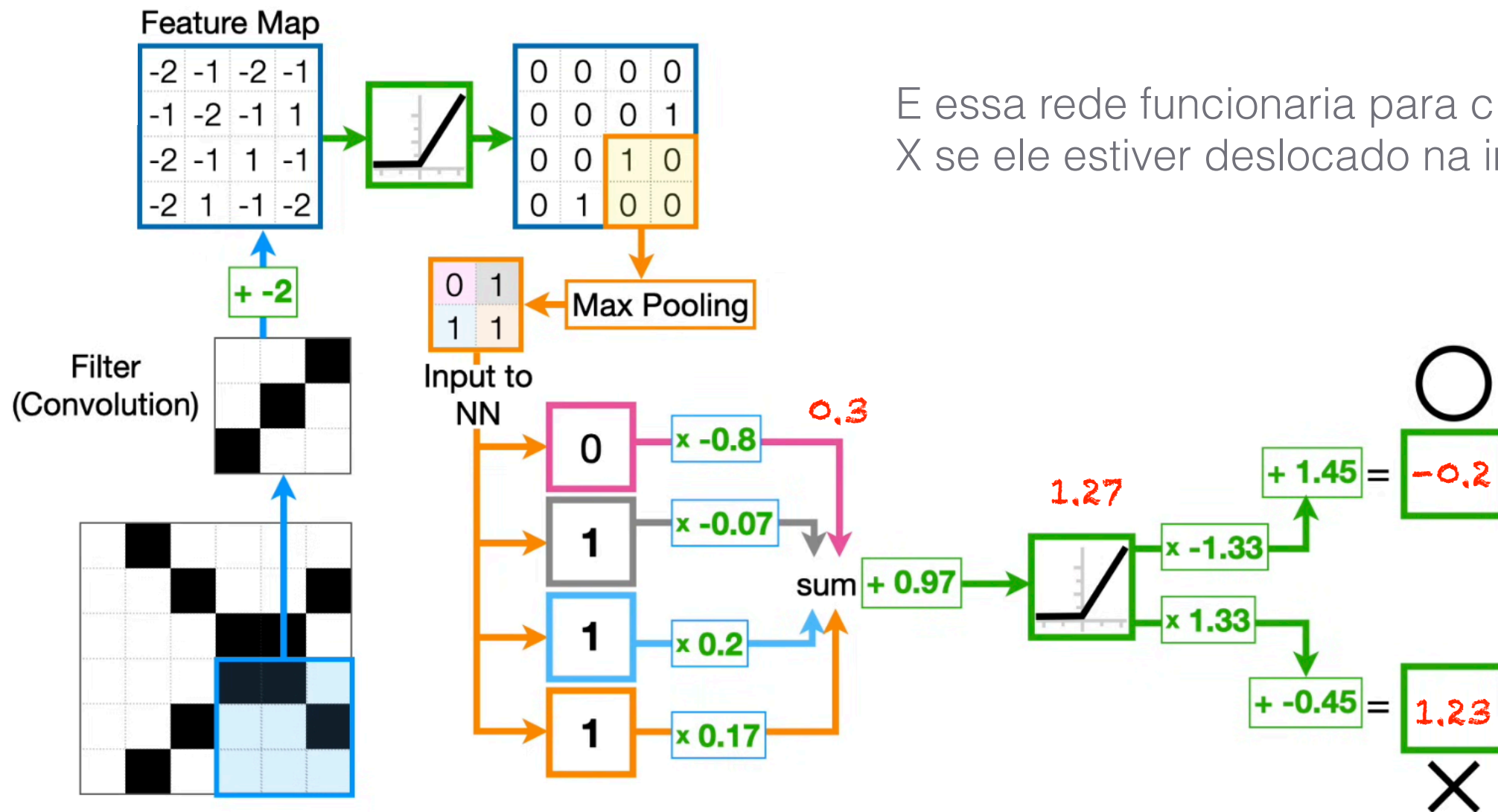


Aqui temos **4 input nodes**, uma única **camada oculta** utilizando **ReLU** como função de ativação e **dois possíveis outputs**

Camadas de Classificação da Rede Neural

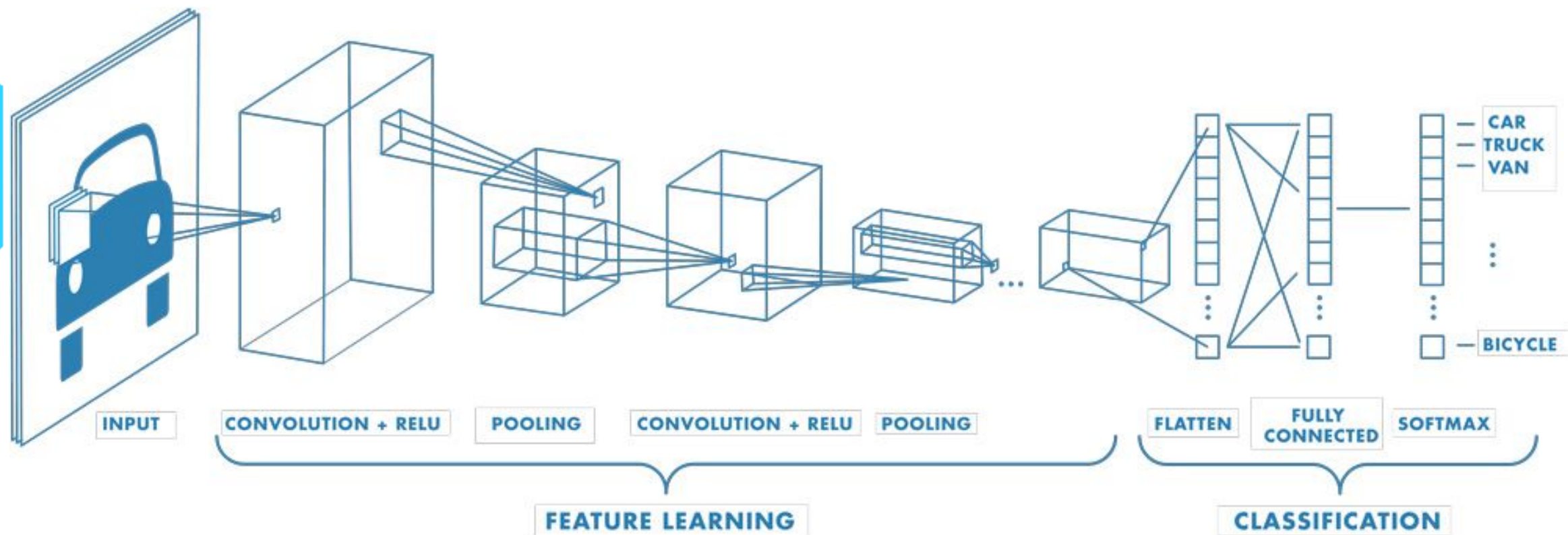


Camadas de Classificação da Rede Neural



E essa rede funcionaria para classificar o X se ele estiver deslocado na imagem?

Classification



LeNet-5 vs AlexNet vs VGGNet

Arquitetura	Ano	Camadas Convolucionais	Tamanho dos Filtros	Parâmetros Totais	Inovação Principal
LeNet-5	1998	3 (S2, C3, C5)	5 × 5	~60 Mil	Primeira CNN comercial (dígitos bancários) [GeeksforGeeks] (https://www.geeksforgeeks.org/machine-learning/convolutional-neural-network-cnn-architectures/)
AlexNet	2012	5	11 × 11, 5 × 5, 3 × 3	~60 Milhões	Uso de GPUs, função de ativação ReLU e Dropout
VGG-16	2014	13	Apenas 3 × 3	~138 Milhões	Filtros pequenos empilhados gerando mesma recepção com menos pesos

A intuição de filtros 3×3 empilhados (Filosofia VGG)

Empilhar **duas camadas de 3×3** tem o mesmo campo receptivo efetivo de uma única camada de **5×5**, mas com uma vantagem matemática: menos parâmetros.

Enquanto 1 filtro de 5×5 tem **25 parâmetros**, 2 filtros de 3×3 possuem apenas $2 \times 9 = 18$ **parâmetros**. Além disso, introduz-se duas não-linearidades (ReLU) em vez de apenas uma.

Função de Perda

- A função de perda quantifica o erro da rede neural.
- Quanto menor a perda, mais próximas as previsões estão dos valores reais.
- Durante o treinamento, o objetivo é minimizar essa perda.
- Em tarefas de classificação, a função mais utilizada é a Cross Entropy.

Por exemplo:

Classe real: Gato

Predição:

Gato = 0.90

Cachorro = 0.10

→ Perda baixa

Classe real: Gato

Predição:

Gato = 0.05

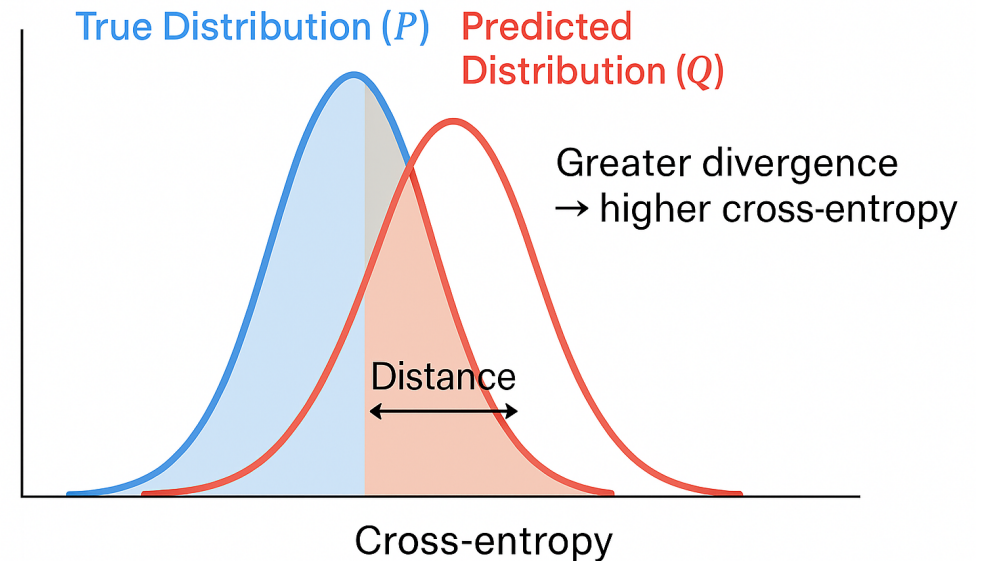
Cachorro = 0.95

→ Perda alta

Cross Entropy

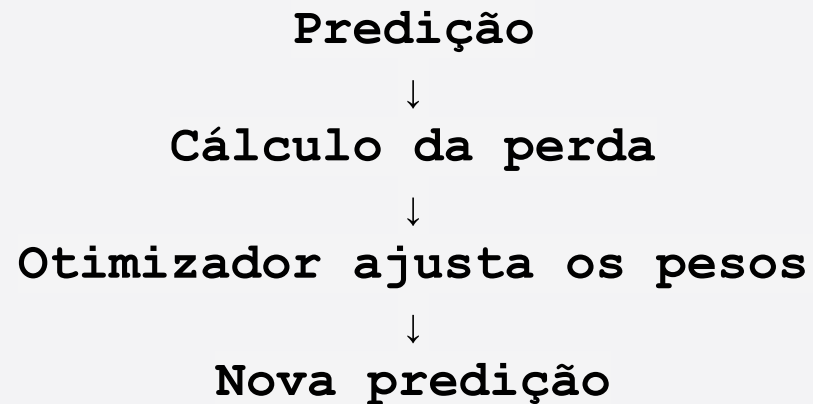
- Penaliza previsões incorretas;
- Penaliza ainda mais previsões incorretas feitas com alta confiança;
- Quanto maior a probabilidade atribuída à classe correta, menor a perda.

$$H(P^* | P) = - \sum_i \underbrace{P^*(i)}_{\text{TRUE CLASS DISTIRBUTION}} \log \underbrace{P(i)}_{\text{PREDICTED CLASS DISTIRBUTION}}$$



Como reduzir a perda?

- Após calcular a perda, os pesos da rede são ajustados;
- O algoritmo responsável por esse ajuste é chamado de **otimizador**;
- O objetivo é encontrar pesos que minimizem a função de perda.



Otimizadores

SGD	Adam
Mais simples	Mais sofisticado
Menor consumo de memória	Maior consumo
Pode exigir mais ajuste manual	Funciona bem em muitos cenários
Muito utilizado em pesquisa	Muito utilizado na prática

Adam é atualmente uma das escolhas mais populares para treinamento de CNNs.

Generalização

Um modelo deve funcionar bem em dados nunca vistos.

Generalização é a capacidade de aplicar o conhecimento aprendido a novos exemplos.

Bons resultados apenas no conjunto de treino não garantem um modelo útil.

Treino = 99%

Teste = 98%

Boa generalização

Treino = 99%

Teste = 65%

Má generalização

Overfitting

Um modelo deve funcionar bem em dados nunca vistos.
Generalização é a capacidade de aplicar o conhecimento aprendido a novos exemplos.
Bons resultados apenas no conjunto de treino não garantem um modelo útil.

Treino = 99%

Teste = 98%

Boa generalização

Treino = 99%

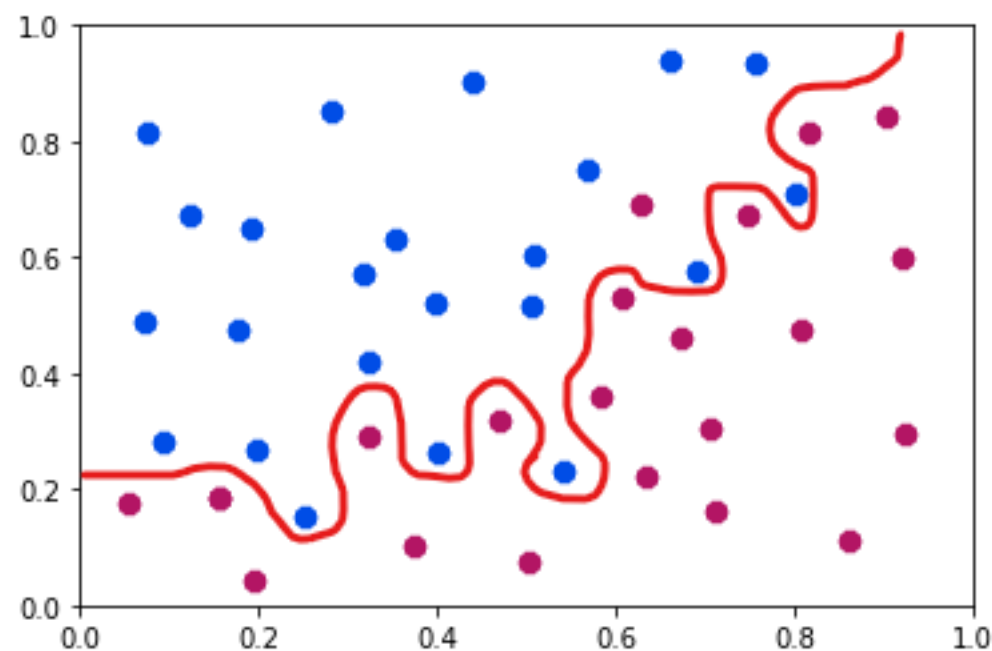
Teste = 65%

Má generalização

Overfitting

Ocorre quando o modelo aprende detalhes específicos do conjunto de treino.

O desempenho no treino continua aumentando, No entanto, o desempenho em dados novos começa a piorar.



Como Mitigar Overfitting?

- Simplificar a arquitetura;
- Data Augmentation;
- Early Stopping;
- Dropout;
- Regularization.

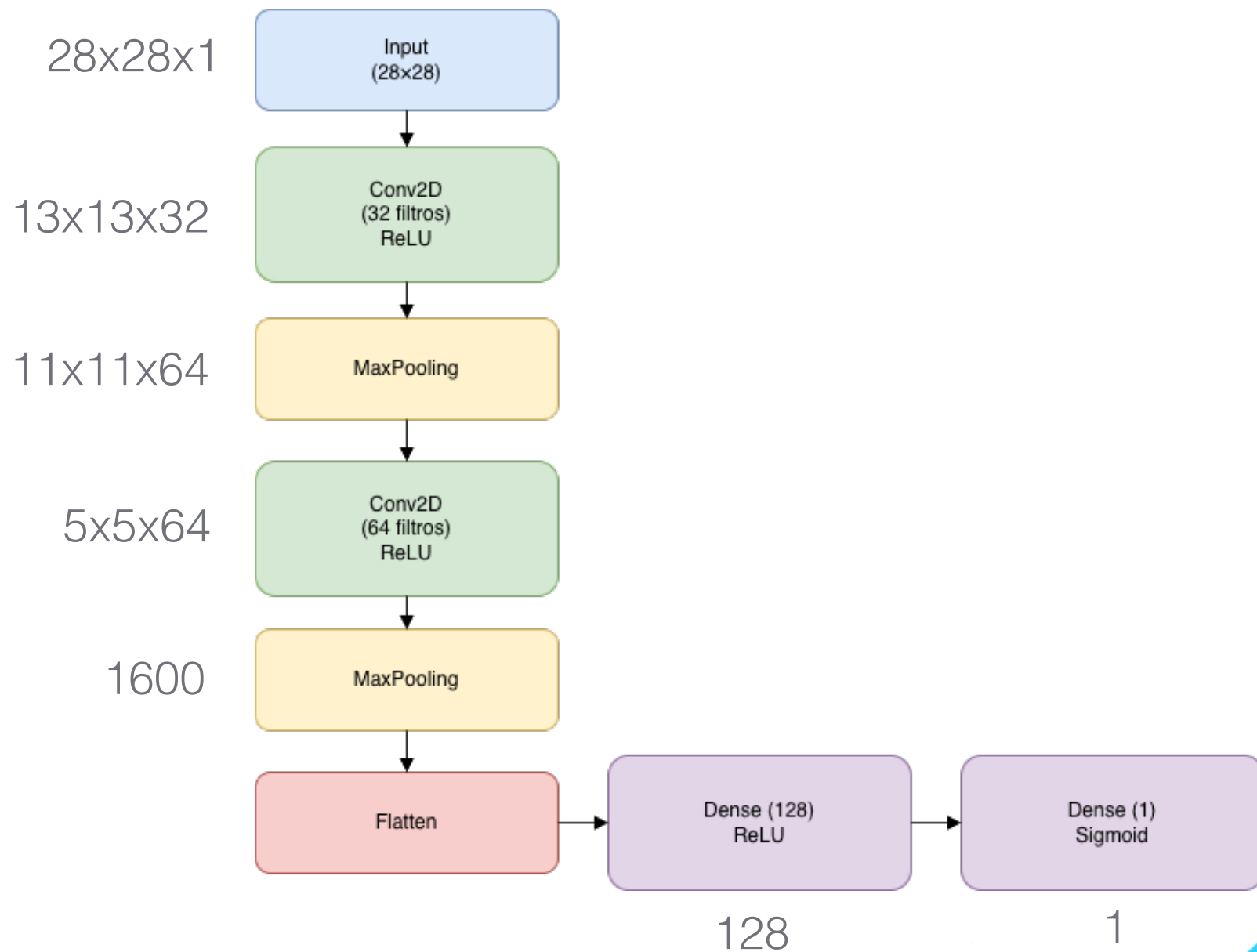
Prática

Utilizando o dataset Fashion-MNIST (disponível no keras) vamos criar um classificador binário utilizando Redes Neurais Convolucionais.

Vamos utilizar somente as classes 7 e 9.



Prática



Desafio Prático (1/2)

Desenvolva uma Rede Neural Convolucional (CNN) capaz de classificar imagens coloridas utilizando o dataset **CIFAR-10**.

O dataset original possui 10 categorias. Para este trabalho, utilize apenas as seguintes classes:

- 0 - Airplane (Avião)
- 1 - Automobile (Automóvel)
- 8 - Ship (Navio)
- 9 - Truck (Caminhão)

O desafio é adaptar o código que vimos anteriormente para uma classificação multiclasse com imagens coloridas. A teoria que vimos segue a mesma, só será necessário trabalhar com funções que usem convoluções 3D, adaptar as funções de ativação utilizadas e a função de perda

Desafio Prático (2/2)

Utilize diferentes arquiteturas (pelo menos 3) na CNN para avaliar como elas impactam no resultado da classificação.

Descreva as percepções obtidas ao testá-las e encaminhe juntamente do código para <gabrieli.muller@acad.ufsm.br> até 26/06.

O trabalho pode ser feito individualmente ou em dupla.

O QUE SÃO E COMO FUNCIONAM AS

Redes Neurais Convolucionais (CNNs)

Prof. Dr. Luis Alvaro de Lima Silva
Mestranda Gabrieli S. J. Müller